

Reviewer Pack — Minimal Norm of Characteristic Functions and Influence Compl...

Entry b6ea20e61b71 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 03:19:18 UTC

Minimal Norm of Characteristic Functions and Influence Complexity

Entry ID: b6ea20e61b71 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 03:19:18 UTC

1. Conjecture

Field A (mathematical branch): Functional Analysis **Field B** (complexity object): Boolean Function Complexity Theory

Statement:

{‘quantitative_relation’: ‘ $E[X(\text{influence}(\chi(\text{instance})))] = \Theta(\|\chi(\text{instance})\|^2)$ ’, ‘counterexample_format’: ‘For any instance with $\chi(\text{instance})$ having a norm of 1, if influence complexity is less than the square of its norm, then the conjecture is falsified.’}

Rationale (proposer’s reasoning):

{‘explanation’: ‘Functional analysis provides tools to study the structure and properties of vector spaces, including norms. The characteristic functions of boolean functions provide a way to represent these functions in a space with a well-defined norm. If there exists a direct relationship between the norm of the characteristic function and the influence complexity, it could reveal a deep connection between functional analysis and complexity theory.’}

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d4b26ce4dee98ab5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the mean squared correlation coefficient between influence complexity and the square of the characteristic function’s norm across all seeds exceeds 0.7, the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "functional analysis" AND "Boolean function complexity theory" AND "influence complexity" - "characteristic functions" in title AND ($\chi(\text{instance})^2$) in abstract - "minimal norm" in title AND influence complexity in abstract AND functional analysis

Top relevant hits considered: - [http://arxiv.org/abs/2112.06776v2] Keyphrase Generation Beyond the Boundaries of Title and Abstract - [http://arxiv.org/abs/2604.08602v1] TiAb Review Plugin: A Browser-Based Tool for AI-Assisted Title and Abstract Screening - [http://arxiv.org/abs/1810.12657v1] The effect of a country's name in the title of a publication on its visibility and citability - [http://arxiv.org/abs/2306.01503v2] Strategies with minimal norm are optimal for expected utility maximization under high model ambiguity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_characteristic_function(f):
        n = int(math.log2(len(f)))
        char_func = [0] * (2**n)
        for i in range(len(f)):
            char_func[i] = f[i]
        return char_func

    def compute_norm(v):
        sum_squares = sum(x**2 for x in v)
        return math.sqrt(sum_squares)

    def compute_influence_complexity(f):
        n = int(math.log2(len(f)))
        influence = 0
        for i in range(n):
            for j in range(1 << i):
                f_tilde = [f[j] if (j & (1 << k)) == 0 else 1 - f[j] for k in range(n)]
```

```

        diff = sum(abs(f[k] - f_tilde[k]) for k in range(len(f)))
        influence += diff
    return influence / len(f)

n_values = [5, 10, 15, 20, 30, 40]
total_influence_complexity = 0
total_norm_squared = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        f = generate_boolean_function(n)
        char_func = compute_characteristic_function(f)
        norm = compute_norm(char_func)
        influence_complexity = compute_influence_complexity(f)

        if norm == 1:
            total_influence_complexity += influence_complexity
            total_norm_squared += norm ** 2
            instances_tested += 1

if instances_tested == 0:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instance found"
    }

correlation_coefficient = total_influence_complexity / instances_tested
return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "conjecture_holds": correlation_coefficient > 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:

```

```

print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    counterexample = "Influence complexity less than the square of the norm"
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ace6dac.py", line 86, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ace6dac.py", line 55, in run_trial
    influence_complexity = compute_influence_complexity(f)
                          ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ace6dac.py", line 41, in compute_influ
    diff = sum(abs(f[k] - f_tilde[k]) for k in range(len(f)))
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ace6dac.py", line 41, in <genexpr>
    diff = sum(abs(f[k] - f_tilde[k]) for k in range(len(f)))
                ~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide a result for the mean squared correlation coefficient. | next: Investigate and fix the crash in the test code to ensure it can run to completion and produce the required data.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13915
2	preregistration	ollama_remote	glm4:latest	0	0	8752
3	novelty	ollama_remote	glm4:latest	0	0	8242
4	novelty	ollama_remote	glm4:latest	0	0	8886

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23286
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11707
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11202
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9964
9	judge	ollama_remote	glm4:latest	0	0	24847

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120801 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/b6ea20e61b71.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b6ea20e61b71.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b6ea20e61b71.tar.gz (if generated)